



Chapitre 3 : Synchronisation des processus *Threads*



1

1



Plan

- La notion des threads
 - Communication entre threads
 - Synchronisation
 - Verou
 - Semaphore
 - Moniteur
- 

2

2

Plan

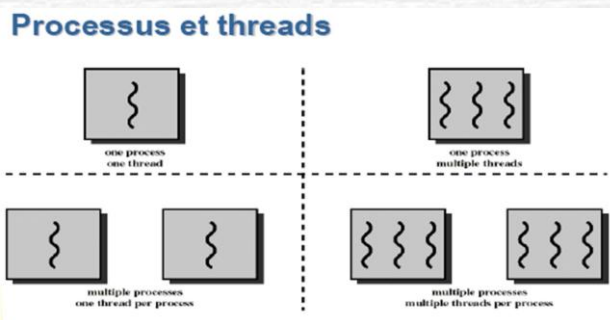
- La notion des threads
- Communication entre threads
- Synchronisation
 - Verou
 - Semaphore
 - Moniteur

3

3

Threads

- De nombreux systèmes d'exploitation modernes offrent la possibilité d'associer à un même processus plusieurs chemins d'exécution (multithreading, multithreading d'exécution).
- Un thread (ou fil d'exécution en français) est une partie du code d'un programme (une fonction), qui se déroule parallèlement à d'autres parties du programme.

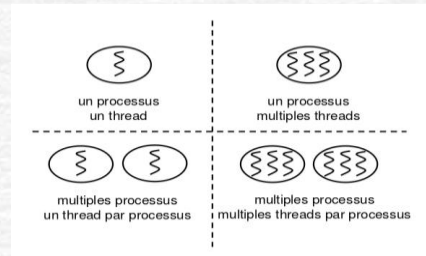


4

4

Threads

- Un thread est une unité d'exécution rattachée à un processus, chargée d'exécuter une partie du processus.
- Un processus est vu comme étant un ensemble de ressources (espace d'adressage, fichiers, périphériques...) que ses threads (flots de contrôle ou processus légers) partagent.
- Lorsqu'un processus est créé, un seul flot d'exécution (thread) est associé au processus. Ce thread peut en créer d'autres. Chaque thread a :
 - un identificateur unique
 - une pile d'exécution
 - des registres (un compteur ordinal)
 - un état...
- Le multithreading permet l'exécution simultanée ou en pseudo-parallèle de plusieurs parties d'un même processus.



5

5

Threads

- Pourquoi utiliser les threads:
 - Améliorer la rapidité des applications (pas de blocages pour des tâches qui peuvent être effectuée en parallèle)
 - Exploiter de manière efficace les machines multiprocesseurs
 - Améliorer la structure de votre programme
- Le premier intérêt peut être d'effectuer un calcul qui dure un peu de temps (plusieurs secondes, minutes, ou heures) sans que l'interface soit bloquée (le programme continue à répondre aux signaux).
 - Le processus MS-Word implique plusieurs threads: interaction avec le clavier, rangement de caractères sur la page, sauvegarde régulière du travail fait
 - contrôle orthographe, etc
- Les fonctions liées aux threads sont dans la bibliothèque **pthread.h**
 - Pour créer un thread, il faut créer une fonction qui va s'exécuter dans le thread, qui a pour prototype : `void *ma_fonction_thread(void *arg);`
 - Dans cette fonction, on met le code qui doit être exécuté dans le thread.

6

6

Threads

```
#include <unistd.h>
#include <pthread.h>
pid_t getpid(void);
pthread_t pthread_self(void);
```

- Création des threads: crée un processus léger qui exécute la fonction **fonction** avec l'argument **arg** et les attributs **attr**, renvoie 0 si l'appel réussit, sinon renvoie l'identifiant de l'erreur

```
#include <pthread.h>
int pthread_create(pthread_t *p_tid, pthread_attr_t attr,
void *(*fonction)(void *arg), void *arg);
```

- **pthread_t *p_tid** : ID du thread
- **const pthread_attr_t *attr** : les attributs du thread (taille de la pile, priorité....), Null par défaut
- **void *(*fonction)(void *)** : La fonction à exécuter
- **void *arg** : liste des paramètres qui seront transmis à la fonction au lancement du thread
- La commande de compilation devient :

```
gcc -lpthread programme.c
```

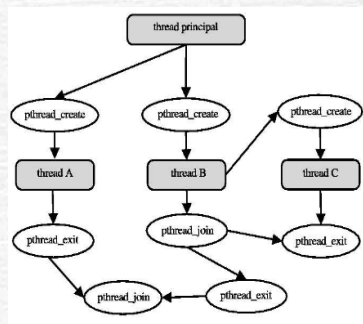
7

7

Threads

- Terminaison d'un thread
 - Le thread appelant se termine avec une valeur de retour égale à ***p_status**
 - Ce code de retour est accessible par les autres threads à travers la fonction **pthread_join()**

```
#include <pthread.h>
int pthread_exit(int *p_status);
```



8

8

Threads

- Exemple de création des threads (creation1.c)

```
#include<stdio.h>
#include<pthread.h>

pthread_t pthread_id[4];

void *f_thread(void *i){
    printf("Je suis le thread %d d'identite %d.%lu \n", (int *)i, getpid(), pthread_self());
}

int main(){
    int i, t;
    printf("Je suis le thread initial %d.%lu \n", getpid(), pthread_self());
    for(i=0; i<4; i++){
        t = pthread_create(&pthread_id[i], NULL, f_thread, (void *)i);
        if(t != 0)
            fprintf( stderr, "Erreur de creation du thread %d \n", i);
        sleep(1);
    }
}
```

```
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./creation1
Je suis le thread initial 104333.124874933610304
Je suis le thread 0 d'identite 104333.124874930517696
Je suis le thread 1 d'identite 104333.124874922124992
Je suis le thread 2 d'identite 104333.124874913732288
Je suis le thread 3 d'identite 104333.124874905339584
```

9

9

Threads

- Exemple de création des threads (creation.c)

```
#include<stdio.h>
#include<pthread.h>

static void *task_a(void *p_data){
    puts("Hello World A");
    return NULL;
}

static void *task_b(void *p_data){
    puts("Hello World B");
    return NULL;
}

int main (void){
    pthread_t ta;
    pthread_t tb;

    puts("main init");

    pthread_create(&ta, NULL, task_a, NULL);
    pthread_create(&tb, NULL, task_b, NULL);
    sleep(1);
    puts("main end");
    return 0;
}
```

```
saad@NSAAD:~/Desktop/SystProg/Chap3$ gcc -lpthread creation.c -o creation
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./creation
main init
main end
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./creation
main init
Hello World A
main end
```

```
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./creation
main init
Hello World A
Hello World B
main end
```

10

10

Threads

- Exemple : partage des variables

```
#include<stdio.h>
#include<pthread.h>

int n=1, m=2;
pthread_t pthread_id[4];

void *f(void *i){
    int k=0;
    n++;m++;
    printf("Thread %d d'identite %d.%lu (n=%d, m=%d) \n", i, getpid(), pthread_self(), n, m);
    return NULL;
}

int main(){
    int i;
    printf("Je suis le thread initial (n=%d, m=%d) \n", n, m);
    for (i=0; i<4; i++){
        pthread_create(&pthread_id[i], NULL, f, (void *)i);
    }
    sleep(1);
    printf("Je suis le thread initial (n=%d, m=%d) \n", n, m);
    return 0;
}
```

```
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./partage
Je suis le thread initial (n=1, m=2)
Thread 0 d'identite 111618.137318748911296 (n=2, m=3)
Thread 1 d'identite 111618.137318740518592 (n=3, m=4)
Thread 2 d'identite 111618.137318732125888 (n=4, m=5)
Thread 3 d'identite 111618.137318723733184 (n=5, m=6)
Je suis le thread initial (n=5, m=6)
```

11

11

Threads

- Passage des paramètres: crée un thread qui dort un nombre de second passé en argument, pendant que le thread principal attend qu'il se termine.

```
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>
#include<unistd.h>
#include<time.h>

void *ma_fonction_thread(void *arg){
    int nbsec = (int)arg;
    printf("Je suis un thread et j'attends %d secondes \n", nbsec);
    sleep(nbsec);
    puts("Je suis un thread et j'ai fini d'attendre \n");
    pthread_exit(NULL); // Termine le thread proprement
}

int main (void){
    int ret;
    pthread_t my_thread;
    int nbsec;
    time_t t1;
    srand(time(NULL));
    t1 = time(NULL);
    nbsec = rand() % 10 + 1; // On attend entre 1 et 10 secondes
    ret = pthread_create(&my_thread, NULL,
        ma_fonction_thread, (void *)nbsec);
    if (ret != 0){
        fprintf(stderr, "Erreur de creation du thread \n");
        exit(1);
    }
    sleep(15); // Le thread principal attend plus longtemps que le thread pour
    // s'assurer qu'il a fini
    printf("Dans le main, nbsec = %d\n", nbsec);
    printf("Duree de l'operation = %ld secondes\n", time(NULL)-t1);
    return 0;
}
```

```
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./passage1
Je suis un thread et j'attends 4 secondes
Je suis un thread et j'ai fini d'attendre
```

```
Dans le main, nbsec = 4
Duree de l'operation = 18 secondes
```

13

13

Threads

- Passage des paramètres: crée un thread qui lit une valeur entière *et la retourne à la main*

```
#include<unistd.h>
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>
#include<time.h>

void *ma_fonction_thread(void *arg){
    int resultat;
    printf("Je suis un thread. Veuillez entrer un nombre entier : \n");
    scanf("%d", &resultat);
    pthread_exit((void *)resultat);
    // Termine le thread et retourne le résultat
}

int main(void){
    int ret;
    pthread_t my_thread;
    // Crée le thread
    ret = pthread_create(&my_thread, NULL, ma_fonction_thread, NULL);
    if (ret != 0){
        fprintf(stderr, "Erreur de creation du thread \n");
        exit(1);
    }
    sleep(5); // Le thread principal attend que l'utilisateur entre
    // un nombre pour 5 secondes sinon le thread se termine
    printf("Dans le main, retour est %d\n", ret);
    return 0;
}
```

```
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./passage2
Je suis un thread. Veuillez entrer un nombre entier :
5
Dans le main, retour est 0
saad@NSAAD:~/Desktop/SystProg/Chap3$
```

14

14

Threads

- Comparaison des identifiants de threads
 - La fonction retourne 0 si tid1 = tid2
 - Une valeur non-nulle sinon

```
#include <pthread.h>
int pthread_equal(pthread_t tid1, pthread_t tid2);
```

- Terminaison d'un thread: termine l'exécution du thread courant avec une valeur de retour particulière
 - Le thread appelant se termine avec une valeur de retour égale à ***p_status**
 - Ce code de retour est accessible par les autres threads à travers la fonction **pthread_join()**

```
#include <pthread.h>
int pthread_exit(int *p_status);
```

- Avoir le TID d'un thread : **pthread_t pthread_self(void);**

15

15

Plan

- La notion des threads
- **Communication entre threads**
- Synchronisation
 - Verrou
 - Semaphore
 - Moniteur

16

16

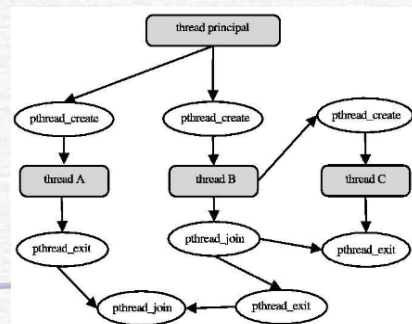
Communication entre threads

- Communication entre les threads: synchronisation sur la terminaison

```
#include <pthread.h>
int pthread_join (pthread_t tid, int ** status);
```

- Permet a un thread d'attendre la terminaison du thread **tid** et de récupérer une valeur de retour correspondant a sa terminaison
- L'appel est bloquant, jusqu'à ce que le thread **tid** exécute **pthread_exit()** ou qu'un autre thread ait demandé son abandon
- Si le thread **tid** est déjà terminé, le retour est immédiat et ****status** contient le code de retour
- Le résultat est 0 en cas de réussite, un code d'erreur non-nul sinon

- Le thread principal crée deux threads **A** et **B** et attend la fin de leurs exécutions. Un de ces threads crée lui-même un thread **C** qu'il attendra avant de terminer.



17

Communication entre threads

- Exemple: Synchronisation sur la terminaison

```
#include<stdio.h>
#include<unistd.h>
#include<pthread.h>

void afficher(int n, char lettre){
    int i, j;
    for(j=1;j<n;j++){
        for(i=1;i<1000000000;i++){
            printf("%c\n", lettre);
        }
    }
}

void *threadA(void* inutilise){
    afficher(10, 'A');
    printf("Fin du thread A\n");
    pthread_exit(NULL);
}

void *threadC(void* inutilise){
    afficher(15, 'C');
    printf("Find du thread C\n");
    pthread_exit(NULL);
}
```

```
void *threadB(void* inutilise){
    pthread_t thC;
    printf("Creation du threadC\n");
    pthread_create(&thC, NULL, threadC, NULL);
    pthread_join(thC, NULL);

    printf("Le thread B attend la fin du threadC\n");
    afficher(10, 'B');

    printf("Fin du thread B\n");
    pthread_exit(NULL);
}

int main(){
    int i;
    pthread_t thA, thB;
    printf("Creation du threadA\n");
    pthread_create(&thA, NULL, threadA, NULL);
    pthread_join(thA, NULL);
    printf("Creation du threadB\n");
    pthread_create(&thB, NULL, threadB, NULL);
    sleep(1);

    // Attendre que les threads aient termine
    printf("Le thread principal attend que les autres se terminent\n");
    pthread_join(thB, NULL);

    return 0;
}
```

18

18

Communication entre threads

```
saad@NSAAD: ~/Desktop/SystProg/Chap1$ gcc -lpthread communication1.c -o communication1
saad@NSAAD: ~/Desktop/SystProg/Chap1$ ./communication1
Creation du threadA
A
A
A
A
A
A
A
A
A
A
Fin du thread A
Creation du threadB
Creation du threadC
C
C
C
C
C
C
C
C
C
C
C
C
C
C
C
Find du thread C
Le thread B attend la fin du threadC
B
B
B
B
B
B
B
B
Fin du thread B
Le thread principal attend que les autres se terminent
```

19

19

Communication entre threads

- Exemple: addition de deux matrices

```
#include<stdio.h>
#include<unistd.h>
#include<pthread.h>
#include<stdlib.h>
#include<time.h>
#define N 4
#define M 10

pthread_t tid[N]; //Vecteur de 4 threads
int A[N][N], B[N][N];
int C[N][N];

void imprime(int matrice[N][N]){
    int i,j;
    for(i=0; i<N-1; i++){
        printf("\n");
        for(j=0; j<N-1; j++){
            printf("%d\t",matrice[i][j]);
        }
    }
}

void *somme_vectorielle(void *arg){
    int i,j;
    i = (int) arg;
    printf("Somme de file%d\n", i);

    for(j=0; j<N-1; j++){
        C[i][j]=A[i][j]+B[i][j];
    }

    pthread_exit(NULL);
}
```

```
int main(void){
    int i,j,semilla;
    // Generation des Matrices
    semilla = time(NULL);
    srand(semilla);

    for(i=0; i<N; i++){
        for(j=0; j<N; j++){
            A[i][j] = rand() % M; // Aleatoire entre 0 et M-1
            B[i][j] = rand() % M;
        }
    }

    for(j=0; j<N; j++){
        printf("Thread file%d\n", j);
        pthread_create(&tid[j], NULL, somme_vectorielle, (void *) j);
    }

    for(i=0; i<N; i++){
        printf("Attendre thread%d\n", i);
        pthread_join(tid[i], NULL);
    }

    printf("Matrice A :\n");
    imprime(A);
    printf("\nMatrice B :\n");
    imprime(B);
    printf("\nMatrice C :\n");
    imprime(C);

    return 0;
}
```

Matrice A :

0	9	4	0
7	0	3	0
3	8	3	1
6	6	5	2

Matrice B :

1	3	5	1
1	3	5	2
6	8	4	3
3	7	1	7

Matrice C :

1	12	9	1
8	3	8	2
9	16	7	4
14	13	6	9

Thread file0
Thread file1
Somme de file0
Thread file2
Somme de file1
Thread file3
Attendre thread0
Somme de file2
Somme de file3
Attendre thread1
Attendre thread2
Attendre thread3

20

20

Communication entre threads

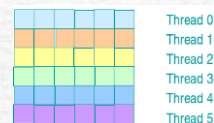
- Exercice: multiplication de deux matrices (voir Lab6)

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \cdot \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{pmatrix} =$$

$$\begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} & a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32} & a_{11}b_{13} + a_{12}b_{23} + a_{13}b_{33} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} & a_{21}b_{12} + a_{22}b_{22} + a_{23}b_{32} & a_{21}b_{13} + a_{22}b_{23} + a_{23}b_{33} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} & a_{31}b_{12} + a_{32}b_{22} + a_{33}b_{32} & a_{31}b_{13} + a_{32}b_{23} + a_{33}b_{33} \end{pmatrix}$$

```
double A[N][N], B[N][N], C[N][N];
int i, j, k;
for(i = 0; i < N; i++)
{
    for(j = 0; j < N; j++)
    {
        double sum = 0.0;
        for(k = 0; k < N; k++)
            sum += A[i][k] + B[k][j];
        C[i][j] = sum;
    }
}
```

- Création de N fils
- Chaque fil résout 1/N de la matrice C
- Attendons la fin des N fils
- La matrice C peut maintenant être utilisée



21

21

Communication entre threads

- Dans un programme qui utilise les threads qui ne sont pas complètement indépendants, ils doivent communiquer entre eux (*com.c*)

```
#include<stdio.h>
#include<unistd.h>
#include<pthread.h>
#include<stdlib.h>

int NTHREADS = 5;
int global = 0;

int increment(int i){
    return i + 1;
}

void *func(void *arg){
    for (int i = 0; i < 1000000000; i++){
        global = increment(global);
    }

    return NULL;
}
```

```
int main(){
    pthread_t thread[NTHREADS];
    int err;
    for (int i=0; i<NTHREADS; i++){
        err = pthread_create(&thread[i], NULL, func, NULL);
        if (err != 0){
            fprintf(stderr, "Erreur de creation du thread %d\n", i);
            exit(1);
        }
    }

    for (int i=NTHREADS-1; i>=0; i--){
        err = pthread_join(thread[i], NULL);
        if (err != 0){
            error(err, "pthread_join");
        }
    }

    printf("Valeur finale de global : %d\n", global);
    return(EXIT_SUCCESS);
}
```

```
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./prob_comm
Valeur finale de global : 125603645
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./prob_comm
Valeur finale de global : 146790832
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./prob_comm
Valeur finale de global : 129500536
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./prob_comm
Valeur finale de global : 120828207
```

- Non seulement la valeur attendue n'est pas atteinte, mais en plus la valeur change d'une exécution du programme à la suivante.
- Problème de la section critique ou exclusion mutuelle

22

22

Plan

- La notion des threads
- Communication entre threads
- Synchronisation**
 - Verou
 - Semaphore
 - Moniteur

23

23

Synchronisation

- Plusieurs processus s'exécutent en pseudo-parallèle ou en parallèle et partagent des objets (mémoires, imprimantes, etc.).
- Le partage d'objets sans précaution particulière peut conduire à des résultats imprévisibles. L'état final dépend de l'ordonnancement des processus.
- La solution au problème s'appelle **synchronisation des processus**.
 - Il faut empêcher les autres processus d'accéder à un objet partagé si un objet est en train d'être utilisé par un processus, ce qu'on appelle d'assurer l'**exclusion mutuelle**.
 - Les situations de ce type, où deux ou plusieurs processus lisent ou écrivent des données partagées et où le résultat dépend de l'ordonnancement des processus, sont qualifiées d'**accès concurrents**.



```
Requête de A :
/*A1*/  if (solde >= 1000)
/*A2*/      solde -= 1000;
else
    printf(" erreur... ");
```

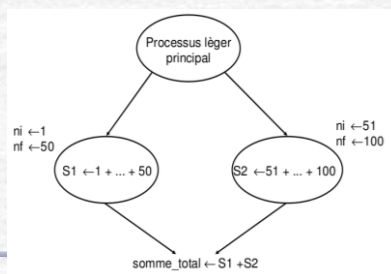
```
Requête de B :
/*B1*/  solde += 100;
```

24

24

Objets et sections critiques

- Objet critique** : Objet qui ne peut être accédé simultanément, les imprimantes, la mémoire, les fichiers, les variables etc
- Section critique** : Ensemble de suites d'instructions qui opèrent sur un ou plusieurs objets critiques et qui peuvent produire des résultats imprévisibles lorsqu'elles sont exécutées simultanément par des processus différents, i.e., une séquence d'instructions qui ne peuvent jamais être exécutées par plusieurs threads simultanément
- Somme des premiers nombres en utilisant des processus légers:
 - si plusieurs processus exécutent concurremment ce code, on peut obtenir un résultat incorrect.
 - La solution correcte nécessite l'utilisation des **sections critiques**.



```
int somme_totale = 0;
void somme_partielle(int ni, int nf)
{
    int j = 0;
    int somme_partielle = 0;
    for (j = ni; j <= nf; j++)
        somme_partielle += j;
    somme_totale += somme_partielle;
    pthread_exit(0);
}
```

25

25

Objets et sections critiques

- Les processus qui exécutent des sections critiques sont structurés comme suit:
 - Section non critique.
 - Demande d'entrée en section critique.
 - Section critique.
 - Demande de sortie de la section critique.
 - Section non critique.
- Conditions sont nécessaires pour réaliser correctement une exclusion mutuelle:
 - Deux processus ne peuvent être en même temps en section critique.
 - Aucune hypothèse ne doit être faite sur les vitesses relatives des processus et sur le nombre de processeurs.
 - Aucun processus suspendu en dehors d'une section critique ne doit bloquer les autres processus.
 - Aucun processus ne doit attendre trop longtemps avant d'entrer en section critique.
- Comment assurer alors une coopération correcte et efficace des processus.
- La bibliothèque POSIX fournit des moyens d'assurer l'exclusion mutuelle, mutex (verrous), et sémaphores

26

26

Mécanismes de synchronisation

- Verrou: les variables de verrouillage
 - réalisation de l'exclusion mutuelle
 - aussi nommé mutex
- Sémaphore
 - gestion de jetons (compteur)
 - associé à une file d'attente
 - exemple : accès multiples (mais bornés) à une ressource
- Moniteur

27

27

Plan

- La notion des threads
- Communication entre threads
- Synchronisation
 - Verrou
 - Semaphore
 - Moniteur

28

28

Les variables de verrouillage

- Utiliser une variable de verrouillage partagée **verrou**, unique, initialisée à 0. Pour rentrer en section critique (voir algorithme de verrouillage), un processus doit tester la valeur du verrou
 - Si elle est égale à 0, le processus modifie la valeur du verrou à 1 et exécute sa section critique, à la fin de la section critique, il remet le verrou à 0
 - Sinon, il attend que le verrou devienne égal à 0, c'est-à-dire :


```
while(verrou !=0)
  while verrou ≠ 0 do
    ; // Attente active
  end while
  verrou ← 1
  Section_critique();
  verrou ← 0
```
- Primitives pour l'utilisation de verrous
 - Création / initialisation / destruction
 - Verrouillage: devenir propriétaire, bloquant
 - Déverrouillage: verrou devient libre

29

29

Les variables de verrouillage

- Le mécanisme **mutex** (MUTual EXclusion) repose sur l'utilisation du type `pthread_mutex_t`
- Un **mutex** est une structure de données qui permet de contrôler l'accès à une ressource. Un mécanisme empêchant deux processus ou plus d'accéder simultanément à une ressource partagée.
- Sur les voies ferrées:
 - un sémaphore empêche deux trains d'entrer en collision sur un tronçon de voie commune.
 - Sur les voies ferrées comme dans les ordinateurs, les sémaphores ne sont qu'indicatifs : si un machiniste ne voit pas le signal ou ne s'y conforme pas, le sémaphore ne pourra éviter la collision.

30

30

Les variables de verrouillage

- Le mécanisme **mutex** (MUTual EXclusion) repose sur l'utilisation du type `pthread_mutex_t`
- Un **mutex** est une structure de données qui permet de contrôler l'accès à une ressource. Un mécanisme empêchant deux processus ou plus d'accéder simultanément à une ressource partagée.
- Un mutex qui contrôle une ressource peut se trouver dans deux états :
 - Libre (ou **unlocked**). Cet état indique que la ressource est libre et peut être utilisée sans risquer de provoquer une violation d'exclusion mutuelle.
 - Réservée (ou **locked**). Cet état indique que la ressource associée est actuellement utilisée et qu'elle ne peut pas être utilisée par un autre thread.
- Les fonctions
 - Initialisation: `pthread_mutex_t mutex = THTREAD_MUTEX_INITIALIZER;`
 - Verrouillage: `int pthread_mutex_lock(pthread_mutex_t * mutex);`
 - Déverrouillage: `int pthread_mutex_unlock(pthread_mutex_t * mutex);`
 - Destruction: `int pthread_mutex_destroy(pthread_mutex_t * mutex);`

31

31

Les variables de verrouillage

- Un mutex peut être vu comme étant une structure de données qui contient deux informations :
 - La valeur actuelle du **mutex** (locked ou unlocked)
 - Une queue contenant l'ensemble des threads qui sont bloqués en attente du **mutex**

```
lock(mutex m) {
    if(m.val==unlocked)
    {
        m.val=locked;
    }
    else
    {
        // Place this thread in m.queue;
        // This thread is blocked;
    }
}
```

```
unlock(mutex m) {
    if(m.queue is empty)
    {
        m.val=unlocked;
    }
    else
    {
        // Remove one thread(T) from m.queue;
        // Mark Thread(T) as ready to run;
    }
}
```

32

32

Les variables de verrouillage

- Exemple: counter (counter.c) avec deux threads

```
saad@NSAAD:~/Desktop/SystProg/Chap3$ ./counter
```

```
Job 0 started and counter value is 0
```

```
Job 1 started and counter value is 0
```

```
Job 1 finished and counter value is 2
```

```
Job 0 finished and counter value is 2
```

- Problème de synchronisation

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<unistd.h>

pthread_t tid[2];
int counter = 0;

void* doSomething(void* arg){
    int iproc = (int) arg;
    counter += 1;
    printf("\n Job %d started and counter value is %d\n", iproc, counter);
    sleep(1);
    printf("\n Job %d finished and counter value is %d\n", iproc, counter);

    return NULL;
}

int main(){
    int i = 0;
    while(i < 2){
        pthread_create(&tid[i], NULL, doSomething, (void*) i);
        i++;
    }

    pthread_join(tid[0], NULL);
    pthread_join(tid[1], NULL);

    return 0;
}
```

33

33

Les variables de verrouillage

- Exemple: counter (counter1.c) avec deux threads

```
saad@NSAAD:~/Desktop/SystProg/Chap1$ ./counter1
Job 0 started and counter value is 1
Job 0 finished and counter value is 1
Job 1 started and counter value is 2
Job 1 finished and counter value is 2
```

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<unistd.h>

pthread_t tid[2];
int counter = 0;
pthread_mutex_t lock;

void* doSomething(void* arg){
    int iproc = (int) arg;
    pthread_mutex_lock(&lock);
    counter += 1;
    printf("\n Job %d started and counter value is %d\n", iproc, counter);
    sleep(1);
    printf("\n Job %d finished and counter value is %d\n", iproc, counter);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(){
    int i = 0;

    while(i < 2){
        pthread_create(&tid[i], NULL, doSomething, (void*) i);
        i++;
    }

    pthread_join(tid[0], NULL);
    pthread_join(tid[1], NULL);

    return 0;
}
```

34

34

Les variables de verrouillage

- Exercice 1 (Lab6): écrire un programme permettant d'additionner N nombres, mais en forme parallèle avec deux threads

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

int main(int argc, char *argv[] )
{
    double sum;
    int N, i;
    N = atoi(argv[1]) ; // Nombre
    sum = 0 ; // somme
    for ( i = 1 ; i<=N; ++ i )
    {
        sum += i ;
    }
    printf("Somme = %10.0f\n" , sum) ;
    return (0) ;
}
```

- Exemple avec deux thread T1 et T2

T1	T2	T1	T2	T1	T2	T1	T2
5	5	7	8	9	3	2	1
i=0	i=1	i=2	i=3	i=4	i=4	i=5	i=6

35

35

Les variables de verrouillage

Exercice 2

- Soit un tableau **M** de **N** éléments rempli par un thread lent et lu par un autre plus rapide.
- Le thread de lecture doit attendre la fin du remplissage du tableau avant d'afficher son contenu. Qui ce que vous remarquez. Comment résoudre ce problème.

Ecriture							
T1	T1	T1	T1	T1	T1	T1	T1
5	5	7	8	9	3	2	1
Lecture							
T2	T2	T2	T2	T2	T2	T2	T2

36

36

Les variables de verrouillage

Exercice 2 (LectureEcritureTab.c)

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h> //sleep
#include<pthread.h>
#define N 5
int M[N] ;

void *lecture( void *arg )
{
    int i ;
    for ( i = 0 ; i < N; i ++ )
        printf( "Lecture : M[%d] = %d\n" , i , M[i] ) ;
    pthread_exit(0) ;
}

void *ecriture( void *arg)
{
    int i ;
    for ( i = 0 ; i < N; i ++ )
    {
        M[i] = i ;
        printf( "Ecriture : M[%d] = %d\n" , i , M[i] ) ;
        sleep(2); //Ralentir le thread d'ecriture
    }
    pthread_exit(0) ;
}

int main( void)
{
    pthread_t th1 , th2 ;
    void *ret ;
    pthread_create(&th1 , NULL , ecriture , NULL) ;
    pthread_create(&th2 , NULL , lecture , NULL) ;
    pthread_join ( th1 , &ret ) ;
    pthread_join ( th2 , &ret ) ;
    return 0 ;
}
```

37

37

Les variables de verrouillage

■ Exercice 2

```
static pthread_mutex_t mutex ;
int M[N] ;

void *lecture( void *arg )
{
    int i ;
    pthread_mutex_lock(&mutex) ;
    for ( i = 0 ; i < N; i ++ )
        printf( "Lecture : M[%d] = %d\n" , i , M[i] ) ;
    pthread_mutex_unlock(&mutex) ;
    pthread_exit(0) ;
}

void *ecriture( void *arg)
{
    int i ;
    pthread_mutex_lock(&mutex) ;
    for ( i = 0 ; i < N; i ++ )
    {
        M[i] = i ;
        printf("Ecriture : M[%d] = %d\n" , i , M[i] ) ;
        sleep(2); //Ralentir le thread d'ecriture
    }
    pthread_mutex_unlock(&mutex) ;
    pthread_exit(0) ;
}

int main (void)
{
    pthread_t th1 , th2 ;
    void *ret ;
    pthread_mutex_init(&mutex , NULL) ;
    pthread_create(&th1 , NULL , ecriture , NULL) ;
    pthread_create(&th2 , NULL , lecture , NULL) ;
    pthread_join ( th1 , &ret ) ;
    pthread_join ( th2 , &ret ) ;
    return 0 ;
}
```

38

38